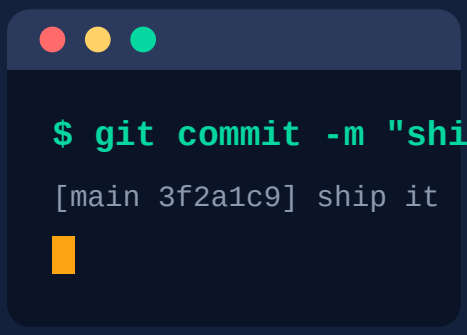

Developer Tools for Software Development

A practical guide to editors, Git, terminals, CI/CD and more

Write • Version • Test • Build • Ship

Updated October 2026

Contents

1. Code editors and IDEs
2. Version control with Git
3. Terminals and command-line tools
4. Languages and package managers
5. Testing, linting and debugging
6. Containers and CI/CD
7. APIs and databases
8. Collaboration, docs and AI assistants
9. Starter toolkits and checklist

> **How to use this guide**
> You do not need every tool here. Pick **one tool per category**, learn it well, and add others only when you hit a real problem. Section 9 has ready-made starter toolkits.

The developer workflow in one line

Write code in an editor, **track** it with Git, **test** it automatically, **package** it (often in a container), **deploy** it through a CI/CD pipeline, and **monitor** it in production. Every tool below fits one of those steps.

Pricing note

Free tiers and licences change often. The "Cost" columns are a general guide; check each vendor's site for current terms.

1. Code editors and IDEs

An **editor** is a lightweight tool for writing code. An **IDE** (integrated development environment) adds a debugger, project system, refactoring and more. The line between them is blurry.

Tool	Best for	Cost	Platforms
Visual Studio Code	Web, Python, Go, Rust and almost anything, via extensions	Free	Win, Mac, Linux
Visual Studio	C#, .NET, C++, Windows and game development	Free Community edition	Windows
JetBrains IDEs (IntelliJ IDEA, PyCharm, WebStorm, Rider)	Deep language support and refactoring	Paid, with some free options	Win, Mac, Linux
Neovim / Vim	Fast keyboard-driven editing, works over SSH	Free	All
Sublime Text	Very fast, light general-purpose editor	Paid licence, free evaluation	All
Android Studio / Xcode	Native Android / iOS and macOS apps	Free	Win, Mac, Linux / Mac

Popular VS Code extensions

- **GitLens**: see who changed each line and why.
- **ESLint and Prettier**: find problems and auto-format JavaScript/TypeScript.
- **Python / Pylance**: IntelliSense and debugging for Python.
- **Docker and Dev Containers**: work inside containers.
- **Live Share**: pair-program in real time.

> **Tip: learn the shortcuts**
> Command palette, go to file, multi-cursor and find-in-project save more time than any plugin. In VS Code the command palette opens with **Ctrl+Shift+P**.

2. Version control with Git

Git records every change to your code so you can undo mistakes, work in parallel branches and collaborate safely. It is the industry standard and the one tool every developer should learn first.

Hosting platforms

- **GitHub**: the largest platform; includes issues, pull requests and Actions.
- **GitLab**: built-in CI/CD; can be self-hosted.
- **Bitbucket**: integrates closely with Jira.

Essential commands

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

git init                        # start a repository
git clone <url>                 # copy a remote repository
git status                      # what changed?
git add .                       # stage all changes
git commit -m "Add login"       # save a snapshot
git switch -c feature/login     # create and move to a new branch
git push -u origin HEAD         # publish the branch
git pull                        # fetch and merge remote changes
git log --oneline --graph       # view history
```

A simple team workflow

- Create a **branch** from main for each feature or fix.
- Make small, focused commits with clear messages.
- Push and open a **pull request** for review.
- Let automated tests run; fix any failures.
- Merge after approval, then delete the branch.

Never commit secrets
Passwords, API keys and tokens must not go into Git. Keep them in environment variables or a secrets manager, and add files like **.env** to **.gitignore**.

3. Terminals and command-line tools

Much of development happens in the terminal: running builds, Git, servers and scripts. A good setup makes this fast and pleasant.

Tool	What it does	Platform
Windows Terminal	Modern tabbed terminal for PowerShell, CMD and WSL	Windows
WSL (Windows Subsystem for Linux)	Run a real Linux environment inside Windows	Windows
PowerShell	Powerful scripting shell, also available on Mac/Linux	All
Bash / Zsh	The standard Unix shells	Mac, Linux, WSL
iTerm2	Feature-rich terminal replacement	Mac
tmux	Split panes and persistent sessions, even over SSH	Mac, Linux, WSL

Handy command-line utilities

Tool	Use it for
`curl`	Send HTTP requests and download files
`jq`	Read and filter JSON from the command line
`ripgrep (rg)`	Very fast text search across a project
`fzf`	Fuzzy-find files, history and branches
`ssh`	Securely connect to remote servers
`make / just`	Save common commands as short tasks

Package managers for tools

Install and update software from the command line: **winget** or **Chocolatey** (Windows), **Homebrew** (Mac and Linux), **apt** (Ubuntu/Debian). Example: `winget install Git.Git`

4. Languages and package managers

Each language has a runtime or compiler plus a **package manager** that downloads libraries and manages dependencies.

Language	Runtime / compiler	Package manager	Typical use
JavaScript / TypeScript	Node.js	npm, pnpm, yarn	Websites, APIs, tooling
Python	CPython	pip, uv, Poetry	Data, AI, scripting, back ends
C# / .NET	The .NET SDK	NuGet	Desktop, web, cloud, games
Java / Kotlin	JDK	Maven, Gradle	Enterprise, Android
Go	Go toolchain	Go modules	Cloud services, CLIs
Rust	rustc	Cargo	Systems, performance-critical code

Manage multiple versions

Different projects often need different language versions. Use a **version manager** so they do not conflict: **nvm** or **fnm** (Node), **pyenv** (Python), **SDKMAN!** (Java), or **asdf / mise** (many languages).

Isolate dependencies

Always keep each project's libraries separate. In Python, create a virtual environment; in Node, rely on the project's `package.json` and lockfile.

```
# Python: create and activate a virtual environment
python -m venv .venv
.venv\Scripts\activate        # Windows
source .venv/bin/activate     # Mac / Linux
pip install requests

# Node.js: start a project
npm init -y
npm install express
```

Commit your lockfile
Files like **package-lock.json**, **poetry.lock** or **uv.lock** make sure everyone on the team installs the same versions.

5. Testing, linting and debugging

These tools catch bugs before your users do. Set them up early, because they are much harder to add to a large project later.

Testing frameworks

Language	Common test tools
JavaScript / TypeScript	Vitest, Jest, Playwright (browser end-to-end), Cypress
Python	pytest, unittest
C# / .NET	xUnit, NUnit, MSTest
Java	JUnit, Mockito
Go / Rust	Built-in `go test` and `cargo test`

Linters and formatters

A **linter** flags suspicious code; a **formatter** rewrites code into a consistent style so reviews focus on logic.

- **JavaScript/TypeScript:** ESLint, Prettier, Biome
- **Python:** Ruff, Black, mypy (type checking)
- **C#:** built-in analyzers, `dotnet format`
- **Any language:** pre-commit hooks that run checks before each commit

Debugging tools

- **Breakpoints and step-through** in VS Code, Visual Studio or JetBrains IDEs.
- **Browser DevTools** (F12): inspect HTML, network requests, console and performance.
- **Profilers** to find slow code or memory leaks.
- **Logging** with levels (debug, info, warning, error) instead of scattered print statements.

> **The testing pyramid**
> Write many fast **unit tests**, a moderate number of **integration tests**, and a few **end-to-end tests** for the most important user journeys.

6. Containers and CI/CD

Docker packages your app and everything it needs into a **container**, so it runs the same on your laptop, a teammate's PC and a server.

Example Dockerfile (Node.js app)

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

```
docker build -t my-app .
docker run -p 3000:3000 my-app
docker compose up        # run app + database together
```

Related tools

- **Docker Compose**: define multi-container setups (app, database, cache) in one file.
- **Kubernetes**: orchestrate many containers at scale. Learn it only when you need it.
- **Terraform / Pulumi**: describe cloud infrastructure as code.

CI/CD pipelines

Continuous Integration automatically builds and tests every change. **Continuous Delivery/Deployment** automatically releases passing changes.

Tool	Notes
GitHub Actions	Configured in YAML inside your repo; huge marketplace of ready-made steps
GitLab CI/CD	Built into GitLab; pipelines defined in `.gitlab-ci.yml`
Jenkins	Self-hosted and highly customizable
Azure DevOps / CircleCI	Hosted pipelines with enterprise features

7. APIs and databases

API clients

Use these to send requests to an API, inspect the responses and save requests for your team.

- **Postman**: full-featured API platform with collections and testing.
- **Insomnia and Bruno**: lighter alternatives; Bruno stores requests as files you can commit to Git.
- **curl and HTTPie**: quick requests from the terminal.
- **Swagger UI / OpenAPI**: describe and document your own API in a standard format.

Databases

Database	Type	Good for
SQLite	Embedded SQL	Small apps, prototypes, desktop and mobile apps
PostgreSQL	Relational (SQL)	General-purpose, reliable production choice
MySQL / MariaDB	Relational (SQL)	Web applications
MongoDB	Document (NoSQL)	Flexible, JSON-like data
Redis	In-memory key-value	Caching, sessions, queues

Database tools

- **DBeaver**: free universal client that connects to most databases.
- **pgAdmin**: management tool for PostgreSQL.
- **DB Browser for SQLite**: simple viewer and editor.
- **Migration tools** (Flyway, Alembic, Entity Framework Migrations, Prisma): version your schema changes like code.

> **Start simple**
> If you are unsure, begin with **SQLite** for learning and **PostgreSQL** for a real project. Both are free and well documented.

8. Collaboration, docs and AI assistants

Project and team tools

Purpose	Tools
Issue and task tracking	GitHub Issues, Jira, Linear, Trello, Azure Boards
Documentation	Markdown files in the repo, Notion, Confluence, MkDocs, Docusaurus
Communication	Slack, Microsoft Teams, Discord
Design handoff	Figma
Diagrams	Mermaid (text-based diagrams in Markdown), draw.io, Excalidraw

AI coding assistants

AI tools can explain code, write tests, suggest fixes and draft boilerplate. Treat their output like code from a new teammate: **review it, run it and test it**.

- **Editor assistants:** GitHub Copilot, Cursor, and JetBrains AI provide suggestions and chat inside your editor.
- **Agentic tools:** Claude Code works in your terminal or IDE, reading your codebase and making multi-file changes at your direction.
- **Chat assistants:** useful for explaining errors, comparing approaches and learning new concepts.

Security basics

- **Dependabot / Renovate:** automatically propose updates for vulnerable dependencies.
- **Snyk, Trivy, npm audit:** scan dependencies and container images for known issues.
- **Secret scanning** (built into GitHub and GitLab) catches leaked keys.
- **Password managers and secret vaults** for credentials; use SSH keys instead of passwords for Git.

9. Starter toolkits and checklist

Recommended starter toolkits

Goal	Suggested toolkit
Web development	VS Code, Git + GitHub, Node.js (npm), Chrome DevTools, Postman, Docker, GitHub Actions
Windows desktop (C#)	Visual Studio Community, .NET SDK, Git + GitHub, xUnit, SQLite, DB Browser for SQLite
Python / data / AI	VS Code or PyCharm, Python + uv or venv, Jupyter, pytest, Ruff, Git + GitHub
Mobile apps	Android Studio (Android), Xcode (iOS), or Flutter / React Native, Git, Firebase
Backend / cloud	VS Code or JetBrains, Docker, PostgreSQL, Redis, Postman, GitHub Actions, Terraform

New-machine setup checklist

- [ ] Install a package manager (winget or Homebrew)
- [ ] Install Git and set your name and email
- [ ] Create an SSH key and add it to GitHub or GitLab
- [ ] Install your editor and key extensions
- [ ] Install the language runtime and a version manager
- [ ] Install Docker Desktop (if you use containers)
- [ ] Set up a terminal with your preferred shell
- [ ] Clone a project and run its tests to confirm everything works

Habits that matter more than tools

- Commit small and often, with clear messages.
- Automate anything you do more than twice.
- Write tests for bugs you fix so they do not return.
- Read error messages carefully; they usually point to the answer.
- Keep documentation (at least a good README) next to the code.
- Update dependencies regularly rather than all at once.

> **Final advice**
> Tools change every year; the underlying ideas (version control, testing, automation, clear communication) do not. Master those and you can pick up any new tool quickly.